Python in Data Science:
Pandas.Series

Pandas

A fast, powerful, flexible and easy to use open source data analysis and manipulation tool, built on top of the Python programming language.

Data I/O
Read/write from CSV, Excel, SQL, HTML.

Statistical Analysis
Calculate correlations, avg, max, and min.

Data Manipulation
Grouping, filtering, and pivot tables.

Integration
Merge, join, and concatenate datasets.

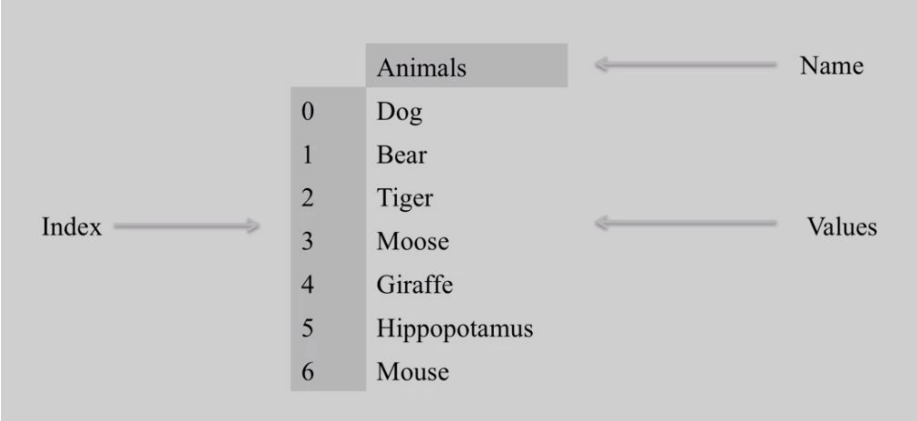
Advanced Filtering
Powerful selection using regular expressions.

Open Source
Built on top of Python for widespread compatibility.

Pandas primary data structures

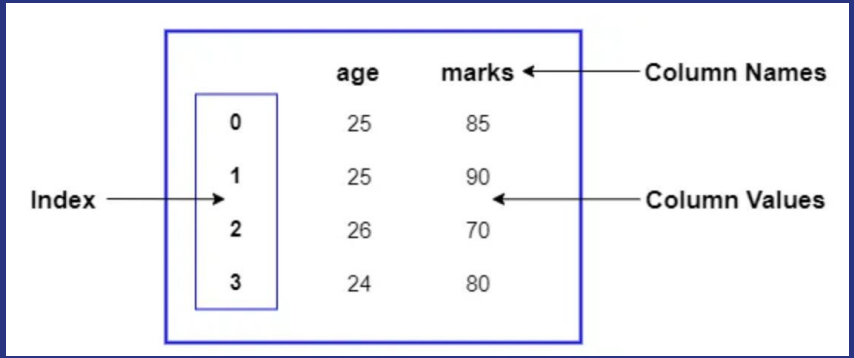
Series

A Series is very similar to a NumPy array



DataFrame

Collection of Series objects combined to share the same index, forming a 2-dimensional table (rows × columns), where all columns share a common row index.



Pandas Series

A Series is very similar to a NumPy array, yet it can be indexed by a label, instead of just a number location

Pandas.Series Creation

```
animals = ["Tiger", "Bear", "Moouse"]
pd.Series(animals, name="Animal")
```

```
0     Tiger
1      Bear
2    Moouse
Name: Animal, dtype: object
```

```
numbers = [1,2,3]
pd.Series(numbers)
```

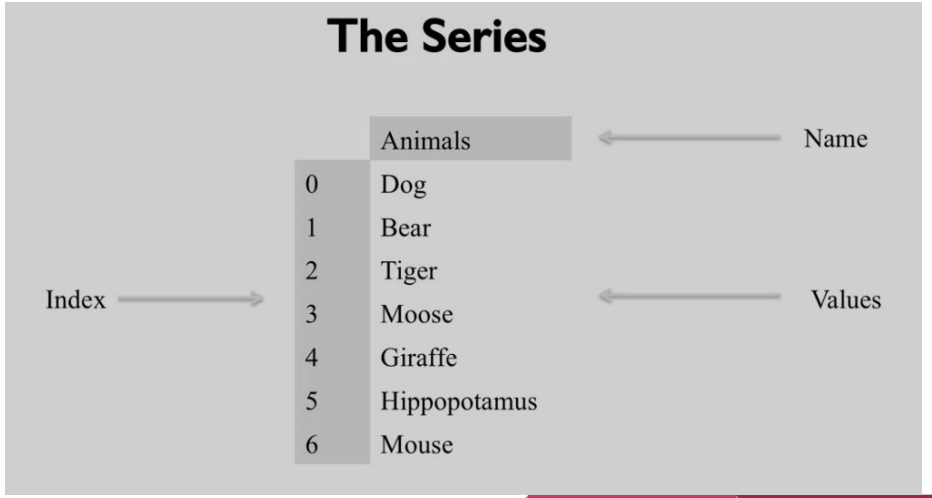
```
0    1
1    2
2    3
dtype: int64
```

```
animals = ["Tiger", "Bear", "Moouse", None]
pd.Series(animals, name="Animal")
```

```
0     Tiger
1      Bear
2    Moouse
3      None
Name: Animal, dtype: object
```

```
person = dict(name="John", age = 20, gender="M")
pd.Series(person)
```

```
name      John
age         20
gender       M
dtype: object
```



Create a Series with Indexes

From a list

- pd.Series([...]) builds a labeled 1-D array.
- A default integer Index (0, 1, 2…) is added.
- .values holds the data; .index the labels.
- Pass index=[...] for custom labels.

series_create.py

```python
import pandas as pd

s = pd.Series([10, 20, 30, 40])
print(s)
# 0    10
# 1    20
# 2    30
# 3    40
# dtype: int64
```

series_labeled.py

```python
s = pd.Series([10, 20, 30],
              index=['a', 'b', 'c'])

print(s['b'])     # 20  (by label)
print(s.index)    # ['a' 'b' 'c']
print(s.values)   # [10 20 30]
```

Automatic typed

```
animals = ["Tiger", "Bear", "Moouse"]
pd.Series(animals, name="Animal")
```

```
0     Tiger
1      Bear
2    Moouse
Name: Animal, dtype: object
```

```
numbers = [1,2,3]
pd.Series(numbers)
```

```
0    1
1    2
2    3
dtype: int64
```

```
animals = ["Tiger", "Bear", "Moouse", None]
pd.Series(animals, name="Animal")
```

```
0     Tiger
1      Bear
2    Moouse
3      None
Name: Animal, dtype: object
```

```
numbers = [1,2,3, None]
pd.Series(numbers)
```

```
0    1.0
1    2.0
2    3.0
3    NaN
dtype: float64
```

Automatic Typing

Pandas infers dtype

- Integers → int64, decimals → float64.
- Text → object; True/False → bool.
- Mixed types fall back to object.
- Check it any time with .dtype.

auto_typed.py

```
pd.Series([1, 2, 3])          # dtype: int64
pd.Series([1.0, 2.5])         # dtype: float64
pd.Series(['a', 'b'])         # dtype: object
pd.Series([True, False])      # dtype: bool

# Mixed types fall back to object
pd.Series([1, 'two', 3.0])    # dtype: object
```

Query a Series

Search and select

- s['label'] returns one value by its index label.
- Pass a list of labels to select several.
- A boolean condition filters like NumPy.
- 'label' in s tests membership.

query_series.py

```
s = pd.Series([10, 20, 30, 40],
              index=['a', 'b', 'c', 'd'])

s['b']              # 20
s[['a', 'c']]       # a 10, c 30
s[s > 20]           # c 30, d 40
'c' in s            # True
```

loc vs iloc

Label vs position

- .iloc[] accesses by position (0-based).
- .loc[] accesses by index label.
- Both support slicing and lists.
- .loc slices are inclusive of the end label.

access_iloc.py

```
s = pd.Series([10, 20, 30],
              index=['a', 'b', 'c'])

# Access by LOCATION (position)
s.iloc[0]     # 10
s.iloc[-1]    # 30
s.iloc[0:2]   # a 10, b 20
```

access_loc.py

```
s = pd.Series([10, 20, 30],
              index=['a', 'b', 'c'])

# Access by INDEX (label)
s.loc['a']        # 10
s.loc['c']        # 30
s.loc['a':'b']    # a 10, b 20
```

Access by location

```
country = {"sg": "singapore","kh": "Cambodia","us": "United States"}
s = pd.Series(country)
s
```

✓ 0.0s

```
sg        singapore
kh         Cambodia
us    United States
dtype: str
```

```
# By location
s.iloc[2]
```

✓ 0.0s

```
'United States'
```

```
# By index
s.loc["us"]
```

✓ 0.0s

```
'United States'
```

Access by index

```
country = {"sg": "singapore","kh": "Cambodia","us": "United States"}
s = pd.Series(country)
s
```

```
sg        singapore
kh         Cambodia
us    United States
dtype: str
```

```
s['kh']
```

```
'Cambodia'
```

Caution with numerics index

Pandas has numerical index

```
import pandas as pd
numbers = pd.Series(['one', 'two', 'three'])
numbers
```

```
0      one
1      two
2    three
dtype: object
```

```
numbers[0]
```

```
'one'
```

```
numbers = pd.Series(["one", "two", "three"], index=[2,3,4])
numbers
```

```
2      one
3      two
4    three
dtype: object
```

```
numbers.iloc[0]
```

```
'one'
```

```
numbers[0]
```

```
KeyError                                  Traceback (most re
File /opt/homebrew/anaconda3/lib/python3.11/site-packages/pa
   3652 try:
-> 3653     return self._engine.get_loc(casted_key)
   3654 except KeyError as err:
```

Query / Search a Series

Boolean Indexing & Filtering

You can query or filter elements in a Pandas Series by passing boolean expressions inside square brackets `[ ]`.

- **Conditional Filtering:** Use logical operators to extract matching elements (e.g., **`numbers > 3`**).
- **Arithmetic Expressions:** Combine with operations like modulo to find specific multiples (e.g., **`numbers % 3 == 0`**).
- **Preserves Indices:** The resulting Series retains its original index positions for each filtered row.

```
numbers = pd.Series([1,2,3,4,5,6])
numbers
```

```
0    1
1    2
2    3
3    4
4    5
5    6
dtype: int64
```

```
numbers[numbers>3]
```

```
3    4
4    5
5    6
dtype: int64
```

```
numbers[numbers%3==0]
```

```
2    3
5    6
dtype: int64
```

Aggregation

2 Ways of aggregation

```
numbers = [1,2,3,4,5]
series = pd.Series(numbers)
series
```

```
0    1
1    2
2    3
3    4
4    5
dtype: int64
```

With forloop (Not recommend)

```
total = 0
for n in numbers:
    total += n
total
```

```
15
```

With vertorization / Numpy (Recommend)

```
np.sum(series)
```

```
15
```

Sum / Count / Average / Minimum / Maximum

Sum

```
np.sum(series)
```

```
15
```

Average

```
np.mean(series), np.average(series)
```

```
(3.0, 3.0)
```

Median ¶

```
np.median(series)
```

```
3.0
```

Minimum and Maximum

```
np.min(series), np.max(series)
```

```
(1, 5)
```

Pandas.Series vs List

Series vs List

Why not just a list?

- list * 2 repeats the list; Series * 2 does math.
- Arithmetic on a Series is element-wise.
- Series carry an index for alignment.
- Series plug straight into the rest of Pandas.

series_vs_list.py

```
nums = [1, 2, 3]
s = pd.Series([1, 2, 3])

nums * 2    # [1, 2, 3, 1, 2, 3]  (repeat)
s * 2       # 0 2, 1 4, 2 6       (math!)

nums + [10] # append
s + 10      # adds 10 to each element
```

Add vs Concat

Aligning two Series

- + aligns on the index — unmatched labels become NaN.
- pd.concat([a, b]) stacks values and keeps all labels.
- Use + for math, concat to combine.
- Mind the index — it drives both.

plus_operator.py

```
a = pd.Series([1, 2], index=['x', 'y'])
b = pd.Series([3, 4], index=['y', 'z'])

# + aligns on the index (NaN where missing)
a + b
# x    NaN
# y    5.0
# z    NaN
```

concat_series.py

```
a = pd.Series([1, 2], index=['x', 'y'])
b = pd.Series([3, 4], index=['y', 'z'])

# concat stacks values, keeps every index
pd.concat([a, b])
# x 1, y 2, y 3, z 4
```

Plus operator

```
n = [1,2,3,4,5]
m = [6,7,8,9,10]
n + m
```

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

```
n = pd.Series([1,2,3,4,5])
m = pd.Series([6,7,8,9,10])
n + m
```

```
0     7
1     9
2    11
3    13
4    15
dtype: int64
```

Use Concat for Pandas

```
nm = pd.concat((n, m))
nm
```

```
0     1
1     2
2     3
3     4
4     5
0     6
1     7
2     8
3     9
4    10
dtype: int64
```

Pandas can be duplicated indexes

```
]: n = pd.Series([1,2,3,4,5])
   m = pd.Series([6,7,8,9,10])
   nm = pd.concat((n, m))
   nm
```

```
]: 0     1
   1     2
   2     3
   3     4
   4     5
   0     6
   1     7
   2     8
   3     9
   4    10
   dtype: int64
```

Access by index can be multiple value

```
nm[0]
```

```
0     1
0     6
dtype: int64
```

Pandas Series Practice Exercises

PYTHON IN DATA SCIENCE · PANDAS

Fill-in-the-blank drills on Series creation, typing, selection, loc/iloc, aggregation, and alignment.

Slash Foundry Co Ltd · August 2026

Creating a Series

SECTION 1 OF 8 · SERIES CREATION

Building a Series from a list, and reading back its values and labels.

Building a Series

1 Create a Series

- Create a Series from
- [10, 20, 30] (default index)

```
import pandas as pd

s = pd.___([10, 20, 30])
print(s)

# hint: the Pandas class that
# builds a labeled 1D array
```

Expected output:

```
[1] → 0   10
      1   20
      2   30
      dtype: int64
```

2 Custom Labels

- Create the same values, but
- label them ['a', 'b', 'c']

```
s = pd.Series([10, 20, 30], ___=['a', 'b', 'c'])
print(s)

# hint: the keyword argument
# that sets custom labels
```

Expected output:

```
[1] → a   10
      b   20
      c   30
      dtype: int64
```

Series Attributes

3 Just the Values

- Get only the underlying
- values of s (no labels)

```
s = pd.Series([10, 20, 30], index=['a', 'b', 'c'])
print(s.___)

# hint: returns a plain NumPy
# array of the data only
```

Expected output:

[1] → [10 20 30]

4 Just the Labels

- Get only the labels
- (not the data) of s

```
s = pd.Series([10, 20, 30], index=['a', 'b', 'c'])
print(s.___)

# hint: the attribute that
# holds the row labels
```

Expected output:

[1] → Index(['a', 'b', 'c'], dtype='object')

Query a Series

SECTION 3 OF 8 · SELECTING DATA

Selecting by a single label, multiple labels, a condition, or testing membership.

Selecting by Label

1 Access by Label

- From s (labels a,b,c),
- get the value labeled 'b'

```
s = pd.Series([10, 20, 30], index=['a', 'b', 'c'])
print(s[___])

# hint: index with the label
# as a string
```

Expected output:

[1] → 20

2 Multiple Labels

- Select the values labeled
- 'a' and 'c' together

```
s = pd.Series([10, 20, 30], index=['a', 'b', 'c'])
print(s[___])

# hint: pass several labels
# at once, as a list
```

Expected output:

[1] → a 10
c 30
dtype: int64

Filtering & Membership

3 Boolean Filter

- Select only the values
- greater than 15

```
s = pd.Series([10, 20, 30], index=['a', 'b', 'c'])
print(s[s ___ 15])

# hint: filters just like a
# NumPy boolean mask
```

Expected output:

[1] → b 20
c 30
dtype: int64

4 Membership Test

- Check whether the label
- 'a' exists in s

```
s = pd.Series([10, 20, 30], index=['a', 'b', 'c'])
print('a' ___ s)

# hint: the same keyword you'd
# use to test list membership
```

Expected output:

[1] → True

loc vs iloc

SECTION 4 OF 8 · POSITION VS LABEL

Two accessors for the same data - by position, and by label.

Position vs Label Access

1 Position Access

- From s (labels a,b,c), get
- the 1st element by position

```
s = pd.Series([10, 20, 30], index=['a', 'b', 'c'])
print(s.___[0])

# hint: position-based accessor,
# 0-based like a Python list
```

Expected output:

[1] → 10

2 Label Access

- From s, get the element
- labeled 'b' directly

```
s = pd.Series([10, 20, 30], index=['a', 'b', 'c'])
print(s.___['b'])

# hint: the label-based
# accessor, not position
```

Expected output:

[1] → 20

Slicing Two Ways

3 Position Slice

- Slice the first two elements
- by position (stop excluded)

```
s = pd.Series([10, 20, 30], index=['a', 'b', 'c'])
print(s.iloc[___])

# hint: a normal Python slice -
# the stop index is excluded
```

Expected output:

```
[1] → a   10
      b   20
      dtype: int64
```

4 Label Slice

- Slice from label 'a' through
- label 'b' - inclusive of 'b'

```
s = pd.Series([10, 20, 30], index=['a', 'b', 'c'])
print(s.loc[___])

# hint: with .loc, the end
# label IS included
```

Expected output:

```
[1] → a   10
      b   20
      dtype: int64
```

Ambiguous Numeric Index

5 Numeric Index Caution

- s2 has a numeric index
- [100,101,102] - get position 0 safely

```
s2 = pd.Series([7, 8, 9], index=[100, 101, 102])
print(s2.___[0])

# hint: with numeric labels, a plain
# s2[0] would be ambiguous
```

Expected output:

[1] → 7

Aggregation

SECTION 5 OF 8 · SUMMARY STATISTICS

The same aggregate can be called as a Series method or as a NumPy function.

Basic Aggregates

1 Sum

- Add up every value in s

```
s = pd.Series([10, 20, 30, 40])
print(s.___())

# hint: the total of all values
```

Expected output:

[1] → 100

2 Count

- Count how many values
- are in s

```
s = pd.Series([10, 20, 30, 40])
print(s.___())

# hint: how many entries
# the Series holds
```

Expected output:

[1] → 4

Add vs Concat

SECTION 7 OF 8 · ALIGNING VS STACKING

One aligns on the index and does math; the other just stacks values together.

Combining Two Series

1 Index Alignment

- Add two Series whose
- indexes don't fully match

```
a = pd.Series([1, 2, 3], index=['x', 'y', 'z'])
b = pd.Series([10, 20], index=['x', 'w'])
print(a ___ b)

# hint: unmatched labels become
# NaN, not an error
```

Expected output:

```
[1] → w    NaN
   x   11.0
   y    NaN
   z    NaN
   dtype: float64
```

2 Stack & Keep All

- Combine a and b by stacking
- them, keeping every label

```
a = pd.Series([1, 2, 3], index=['x', 'y', 'z'])
b = pd.Series([10, 20], index=['x', 'w'])
print(pd.___([a, b]))

# hint: takes a list of Series
# and stacks them together
```

Expected output:

```
[1] → x    1
   y    2
   z    3
   x   10
   w   20
   dtype: int64
```

Duplicate Indexes

SECTION 8 OF 8 · NOT ALWAYS UNIQUE

Pandas allows repeated labels - accessing one can return several values.

Duplicate Labels

1 Duplicate Labels

- Create a Series where the
- label 'a' appears twice

```
s = pd.___([1, 2, 3], index=['a', 'a', 'b'])
print(s)

# hint: same class used to
# create any Series
```

Expected output:

```
[1] → a   1
      a   2
      b   3
      dtype: int64
```

2 Access a Duplicate

- Access the values labeled 'a' -
- notice you get back more than one

```
print(s[___])

# hint: the same single-label
# access as always
```

Expected output:

```
[1] → a   1
      a   2
      dtype: int64
```